2027-I

UNAM

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE ESTUDIOS SUPERIORES ARAGÓN



MATERIA

Web 2

TEMA

CRUD Persona

PROFESOR

Mtro. ISC. MIGUEL ANGEL SÁNCHEZ HERNÁNDEZ

Tabla de contenido

1. Inicio 3
2. Actualizacion de Nodejs ..... **¡Error! Marcador no definido.**
3. Instalar modulos 3
4. Estructura de las carpetas y directorios del proyecto 3
5. Modificar index.js 4
6. Agregar .env 5
7. Agregar db.js 5
8. Agregar Persona.js ..... **¡Error! Marcador no definido.**
9. Modificar inicio_router.js 6
10. Modificar inicioController.js 7
11. Agregar index.ejs 8
12. Correr la aplicación ..... **¡Error! Marcador no definido.**

1. Inicio

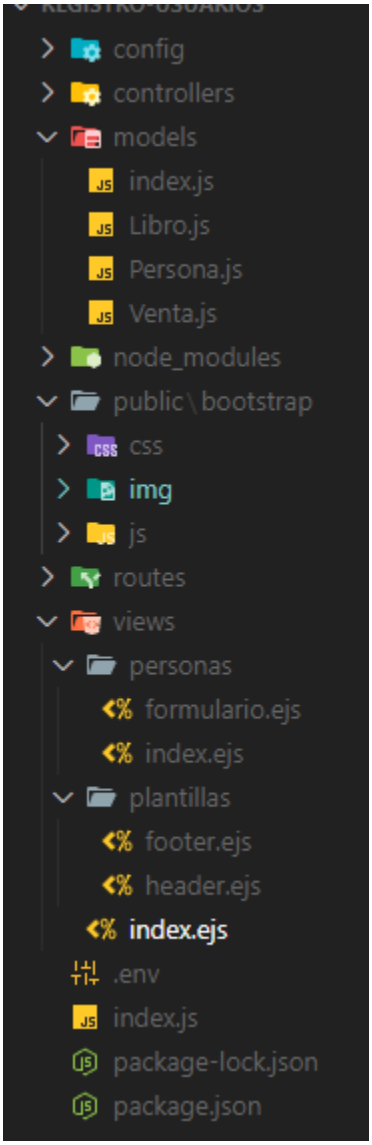
Limpia el esquema **base_web.**

2. Instalar modulos

```
npm i method-override
```

3. Estructura de las carpetas y directorios del proyecto

Sigue la imagen y respeta la estructura, solo cambia o agrega las carpeta y archivos que falten.



4. Modificar index.js

```
import express from "express";
import router from "./routes/inicio_router.js";
import db from "./config/db.js";
import dotenv from 'dotenv';
import methodOverride from "method-override";
import path from 'path';
const __dirname = path.resolve();


dotenv.config({path:'.env'});
//Crear la aplicación
const app = express();

//accesos a los datos del formulario
app.use(express.urlencoded({extended:true}))

// Permite PUT/DELETE vía ?_method=PUT
app.use(methodOverride('_method'));

// Middleware para parsear JSON
app.use(express.json());

// Motor de vistas
app.set('view engine', 'ejs');
//app.set('views', path.join(__dirname, 'views'));
app.set('views', './views');
//carpeta publica
app.use(express.static("public"));

//acceso a bootstrap
app.use('/bootstrap', express.static(
  path.join(__dirname, 'public/bootstrap')
));
//routing
app.use("/personas", router);

//definiendo el puerto
const port = 2800;

(async () => {
  try {
    await db.authenticate();
    console.log('Conexión a la base de datos establecida.');

    await db.sync({ alter: true }); // crea/actualiza tablas
    console.log('Modelos sincronizados.');

    app.listen(port, () => {
```

```
    console.log(`Esperando peticiones en el puerto ${port}`);
   });
  } catch (err) {
   console.error('Error al iniciar:', err);
  }
})();
```

5. De /models crear index.js

```
import Persona from "../models/Persona.js"
import Libro from "../models/Libro.js"
import Venta from "../models/Venta.js"
// Asociaciones
Persona.hasMany(Venta, { foreignKey: 'personaId', as: 'venta' });
Venta.belongsTo(Persona, { foreignKey: 'personaId', as: 'persona' });

Libro.hasMany(Venta, { foreignKey: 'libroId', as: 'venta' });
Venta.belongsTo(Libro, { foreignKey: 'libroId', as: 'libro' });

export { Persona, Libro, Venta }
```

6. Modificar Persona.js

```
import {DataTypes} from 'sequelize';
import db from '../config/db.js';
const Persona = db.define('Persona', {
   id: {
     type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true
   },
   nombre: {
     type: DataTypes.STRING,
     allowNull: false,
   },
   apellido: {
     type: DataTypes.STRING,
     allowNull: false,
   },
   email: {
     type: DataTypes.STRING,
     allowNull: false,
     unique: true,
     validate: { isEmail: true },
   },
   edad: {
     type: DataTypes.INTEGER,
     allowNull: false,
     validate: { min: 0, max: 120 },
```

```
    }
  }, {
    tableName: 'persona',
    timestamps: false,
  });

  export default Persona;
```

7. Crear Libro.js

```
import {DataTypes} from 'sequelize';
import db from '../config/db.js';

const Libro = db.define('Libro', {
    id: {
        type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true
     },
    titulo: {
        type: DataTypes.STRING, allowNull: false
    },
    autor: {
         type: DataTypes.STRING, allowNull: false
         },
    precio: {
        type: DataTypes.DECIMAL(10, 2), allowNull: false
    },
  }, {
     tableName: 'libro',
     timestamps: false
});

export default Libro;
```

8. Crear Venta.js

```
import {DataTypes} from 'sequelize';
import db from '../config/db.js';

const Venta = db.define('Venta', {
  id: {
    type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true
},
  cantidad: {
    type: DataTypes.INTEGER, allowNull: false, defaultValue: 1
},
  fecha: {
    type: DataTypes.DATE, allowNull: false, defaultValue: DataTypes.NOW
    },
}, {
    tableName: 'venta',
    timestamps: false
});
```

```
export default  Venta;
```

9. Modificar inicioController.js

```
import express from "express";
import bcrypt from "bcryptjs";
import {Persona,Libro,Venta} from "../models/index.js"




const inicio = async(req, res) => {
  const persona = await Persona.findAll({ order: [['id', 'DESC']],
    include: [{ model: Venta, as: 'venta' }] });
  res.render("./personas/index.ejs",{ persona, action: '/personas', method: 'POST' })
};


const verCrear = async(req, res) => {
  const persona = await Persona.findAll({ order: [['id', 'DESC']],
    include: [{ model: Venta, as: 'venta' }] });
  res.render("./personas/formulario.ejs",{ persona, action: 'personas/registrar', method: 'POST' })
};

// Guardar persona
const registro = async (req, res) => {
  try {
    const { nombre, apellido, email, edad } = req.body;
    await Persona.create({ nombre, apellido, email, edad });
    const persona = await Persona.findAll({ order: [['id', 'DESC']],
    include: [{ model: Venta, as: 'venta' }] });
    res.redirect("/personas",{ persona,mensaje:null,erro:null})
  } catch (err) {
    const persona = await Persona.findAll({ order: [['id', 'DESC']],
    include: [{ model: Venta, as: 'venta' }] });
    res.render('./personas/index.ejs', {
      persona,
      action: '/personas/registrar',
      method: 'POST',
      mensaje:null,
      error: err.message });
  }
};
// FORMULARIO EDITAR
const editar= async (req, res) => {
  const persona = await Persona.findByPk(req.params.id);
  if (!persona) return res.redirect("/personas",{ persona,mensaje:null,erro:null})
  res.render('personas/formulario', {
    persona,
    action: `/personas/${persona.id}?_method=PUT`,
    method: 'POST',
  });
```

```
};

// ACTUALIZAR
const actualizar= async (req, res) => {
  try {
    const persona = await Persona.findByPk(req.params.id);
    if (!persona) return res.redirect('/personas');
    await persona.update(req.body);
    res.redirect('/personas');
  } catch (error) {
    res.render('persona/formmulario', {
      persona: { ...req.body, id: req.params.id },
      action: `/personas/${req.params.id}?_method=PUT`,
      method: 'PUT',
      error: error.message,
    });
  }
};

// ELIMINAR
const eliminar=async (req, res) => {
  await Persona.destroy({ where: { id: req.params.id } });
  res.redirect('/personas');
};


export { inicio,verCrear, registro,editar,actualizar,eliminar};
```

10. Agregar inicio_router.ejs

```
import express from "express";
import {inicio,registro,verCrear,editar,actualizar,eliminar} from "../controllers/inicioController.js"

const router=express.Router();
//Routing
router.get('/',inicio);
router.get("/crear",verCrear)
router.post("/registrar",registro)
router.get("/:id/editar",editar)
router.put("/:id",actualizar)
router.delete("/:id",eliminar)
export default router
```

11. index.ejs

Borrar index.ejs de view y en la carpeta view/persona crear un nuevo index.ejs

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Registro de Personas</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 700px; margin: 40px auto; }
```

```
   input { display: block; margin: 8px 0; padding: 8px; width: 100%; }
   button { padding: 10px 20px; background: #007bff; color: white; border: none; cursor: pointer; }
   table { width: 100%; border-collapse: collapse; margin-top: 20px; }
   th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
   .ok { color: green; }
   .err { color: red; }
  </style>
</head>
<body>
  <h1>Registrar Persona</h1>

  <% if (mensaje) { %><p class="ok"><%= mensaje %></p><% } %>
  <% if (error) { %><p class="err"><%= error %></p><% } %>

  <form action="/registro" method="POST">
    <input type="text"   name="nombre"   placeholder="Nombre"   required>
    <input type="text"   name="apellido" placeholder="Apellido" required>
    <input type="email"  name="email"    placeholder="Email"    required>
    <input type="number" name="edad"     placeholder="Edad"     required min="0" max="120">
    <button type="submit">Guardar</button>
  </form>

  <h2>Personas registradas</h2>
  <table>
    <tr><th>ID</th><th>Nombre</th><th>Apellido</th><th>Email</th><th>Edad</th></tr>
    <% personas.forEach(p => { %>
      <tr>
        <td><%= p.id %></td>
        <td><%= p.nombre %></td>
        <td><%= p.apellido %></td>
        <td><%= p.email %></td>
        <td><%= p.edad %></td>
      </tr>
    <% }) %>
  </table>
</body>
</html>
```

12. formulario.ejs

En la carpeta view/persona crear un nuevo formulario.ejs

```
<h1><%= persona ? 'Editar' : 'Nueva' %> Persona</h1>

<% if (typeof error !== 'undefined' && error) { %>
  <div class="alert"><%= error %></div>
<% } %>

<form action="<%= action %>" method="<%= method %>">
  <label>Nombre:</label>
  <input type="text" name="nombre" value="<%= persona ? persona.nombre : '' %>" required>
```

```
  <label>Apellido:</label>
  <input type="text"   name="apellido" value="<%= persona ? persona.apellido : '' %>" required>

  <label>Email:</label>
  <input type="email" name="email" value="<%= persona ? persona.email : '' %>" required>

  <label>Edad:</label>
  <input type="text"   name="edad" value="<%= persona ? persona.edad : '' %>" required>

  <button class="btn btn-primary" type="submit">Guardar</button>
  <a class="btn" href="/personas">Cancelar</a>
</form>

<%- include('../plantillas/footer') %>
```

13. hedaer.ejs

En la carpeta de view/plantillas crear header.js

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Sistema Librería</title>
  <link rel="stylesheet" href="/bootstrap/css/bootstrap.min.css">
</head>
<body>
    <nav class="navbar navbar-expand-lg bg-body-tertiary">
       <div class="container-fluid">
         <a class="navbar-brand" href="#">FES ARAGÓN</a>
         <div class="collapse navbar-collapse" id="navbarSupportedContent">
           <ul class="navbar-nav me-auto mb-2 mb-lg-0">
             <li class="nav-item dropdown">
               <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown" aria-
expanded="false">
                 Personas
               </a>
               <ul class="dropdown-menu">
                 <li><a class="dropdown-item" href="/personas">Lista</a></li>
                 <li><a class="dropdown-item" href="/personas/crear">Crear</a></li>
                 <li><hr class="dropdown-divider"></li>

               </ul>
             </li>
         </div>
       </div>
     </nav>
  <main class="container text-center shadow-lg p-3 mb-5 bg-body-tertiary rounded">
```

14. footer.ejs

En la carpeta de view/plantillas crear footer.js

```
</main>
<footer class="container text-center shadow-lg p-3 mb-5 bg-body-tertiary rounded">
  <img src="/bootstrap/img/fes.png" width="80" height="80"/>
  <p>FES ARAGÓN</p>
</footer>
<script src="/bootstrap/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Correr la aplicación y revisar si en la base de datos se hacen los cambios de las operaciones CRUD.